CONFIG FORMATS

CHEAT SHEETS

TOML, INI, .properties, .env and the Config Files You Edit Every Week

Choosing a Format · TOML 1.1 · Tables · Arrays of Tables · INI · .properties · .env · Env Vars · Config Layering · JSONC · JSON5 · YAML Traps · package.json · tsconfig.json · pyproject.toml · compose.yaml · Workflows · Dotfiles · JSON Schema · jq · yq

22 REFERENCE SHEETS

CONTENTS

CHOOSING A FORMAT

01 Config Formats at a Glance
02 Choosing & Comparing Formats

TOML

03 TOML Basics
04 TOML Tables & Inline Tables
05 TOML Arrays & Arrays of Tables
06 TOML Strings, Dates & Versions

INI, PROPERTIES & ENVIRONMENT

07 INI Files
08 Java .properties Files
09 .env Files
10 Environment Variables & Config Layering

JSON & YAML FOR CONFIG

11 JSON for Config (JSONC & JSON5)
12 YAML for Config — Traps

REAL-WORLD CONFIG FILES

13 package.json
14 tsconfig.json & jsconfig.json
15 pyproject.toml
16 compose.yaml (Docker Compose)
17 GitHub Actions Workflow Files
18 Project Dotfiles

VALIDATION & TOOLING

19 Validating Config
20 Converting & Querying Config

QUICK REFERENCE

21 Quick Reference — Syntax Side by Side
22 Quick Reference — Common Config Files

CFG

Table of Contents

22 sheets · syntax · examples · notes

CHOOSING A FORMAT 01 – 02

01 Config Formats at a Gl…
02 Choosing & Comparin…

TOML 03 – 06

03 TOML Basics
04 TOML Tables & Inline T…
05 TOML Arrays & Arrays …
06 TOML Strings, Dates & …

INI, PROPERTIES & ENVIRONMENT 07 – 10

07 INI Files
08 Java .properties Files
09 .env Files
10 Environment Variables …

JSON & YAML FOR CONFIG 11 – 12

11 JSON for Config (JSON…
12 YAML for Config — Tra…

REAL-WORLD CONFIG FILES 13 – 18

13 package.json
14 tsconfig.json & jsconfi…
15 pyproject.toml
16 compose.yaml (Docker…
17 GitHub Actions Workfl…
18 Project Dotfiles

VALIDATION & TOOLING 19 – 20

19 Validating Config
20 Converting & Querying…

QUICK REFERENCE 21 – 22

21 Quick Reference — Sy…
22 Quick Reference — Co…

Sheet 01

Config Formats at a Glance

JSON · YAML · TOML · INI · `.properties` · `.env` · XML · who uses which

SAME JOB, DIFFERENT RULES

A **config file** holds settings a program reads at start-up. Every format here stores keys and values; they differ in **types** (is `8080` a number or text?), **nesting**, **comments** and how strict the parser is. Most of the time the tool picks the format for you: npm uses JSON, Python packaging uses TOML, Docker Compose and CI use YAML, Java uses `.properties`.

NOTES

- JSON, YAML and XML have full sets of their own in Markup & Notation
- This set covers TOML, INI, `.properties`, `.env` and the files built on them
- **Typed** formats: JSON, YAML, TOML · **strings-only**: INI, `.properties`, `.env`
- Read sheet 02 when you get to choose the format yourself

ONE SETTING, SIX FORMATS

```
# TOML                      # INI
[server]                    [server]
host = "0.0.0.0"            host = 0.0.0.0
port = 8080                 port = 8080
debug = false               debug = false

# .properties               # .env
server.host=0.0.0.0         SERVER_HOST=0.0.0.0
server.port=8080            SERVER_PORT=8080
server.debug=false          SERVER_DEBUG=false

# YAML                      # JSON
server:                     {"server": {"host": "0.0.0.0",
  host: 0.0.0.0               "port": 8080, "debug": false}}
  port: 8080
  debug: false
```

CODE EXPLAINED

- → TOML, YAML and JSON parse `8080` as a number and `false` as a boolean
- → INI, `.properties` and `.env` hand everything back as **text**; your code converts it
- → `.properties` and `.env` flatten nesting into dotted or `UPPER_SNAKE` names
- → Only JSON lacks comments; every other format here allows them

SAME DATA IN…

- ⇄ **XML** `<server host="0.0.0.0" port="8080" debug="false"/>`
- ⇄ **HCL** `server { host = "0.0.0.0" port = 8080 }` (Terraform)
- ⇄ **Env var** `SERVER__PORT=8080` (double underscore = nesting in .NET)

GOTCHA

- ! The same value can be a number in one format and a string in another; check types after a conversion
- ! YAML reads `0.0.0.0` as a string but `1.0` as a float; quote values you mean as text

FEATURE COMPARISON

FORMAT	TYPES	NESTING	COMMENTS	SPEC
JSON	Yes	Deep	No	RFC 8259
YAML	Yes (implicit)	Deep	#	YAML 1.2
TOML	Yes (explicit)	Tables	#	TOML 1.1
INI	Strings	Sections	; #	None
.properties	Strings	Dotted keys	# !	Java docs
.env	Strings	None	#	None (dotenv)

WHO USES WHICH

FORMAT	TYPICAL FILES
JSON	package.json, tsconfig.json, .vscode/*.json
YAML	compose.yaml, workflows, Kubernetes
TOML	pyproject.toml, Cargo.toml, ruff.toml
INI	setup.cfg, .gitconfig, php.ini, tox.ini
.properties	application.properties, gradle.properties
.env	.env, .env.local

⚠ COMMON MISTAKES

- Assuming every format has types; reading `port` from `.env` gives `"8080"`, not `8080`
- Writing comments in a `.json` file a strict parser reads (npm, `JSON.parse`)
- Picking YAML for a deep config just because it looks clean, then fighting indentation errors

✓ TIPS & BEST PRACTICES

- Use the format the tool expects; don't invent a second config file next to it
- Convert strings from INI / `.env` / `.properties` once, at start-up, with validation
- Keep one source of truth and generate other formats from it if you must

QUICK SUMMARY

- typed — JSON, YAML, TOML
- strings-only — INI, `.properties`, `.env`
- comments — everything but JSON
- tool decides — follow its format

Sheet 02

Choosing & Comparing Formats

human-edited vs machine-written · typed vs strings-only · comments · deep nesting · tooling support · decision table

WHEN THE CHOICE IS YOURS

For your own app, ask four questions: **Who edits it** (people or programs)? **How deep** is the data? Do you need **comments**? What do your language and deployment tools read **without a dependency**? People editing flat-ish settings → TOML. Deep data written by programs → JSON. Deployment and CI → YAML because the tools require it. Secrets and per-machine values → environment variables.

NOTES

- Python 3.11+ reads TOML with `tomllib` (standard library)
- Every language reads JSON without extra packages
- YAML needs a library almost everywhere and has the most surprises
- XML is still right for document-like data and schemas (see the XML set)

A TYPICAL SPLIT FOR ONE APP

```
# config.toml — checked in, edited by people
[app]
name = "orders"
log_level = "info"

[database]
pool_size = 10
timeout_seconds = 5

[features]
new_checkout = false

# .env — NOT checked in, one per machine
# DATABASE_URL=postgres://app:secret@db:5432/orders
# STRIPE_KEY=sk_live_...

# compose.yaml / CI workflow — YAML because the tools need it
# package.json / tsconfig.json — JSON because the tools need it
```

CODE EXPLAINED

→ Settings that are the same everywhere go in the checked-in file

→ Secrets and values that change per machine come from the environment

→ Tool configs stay in the tool's format; don't wrap them

→ This split is the 12-factor idea covered on sheet 10

SAME DATA IN…

⇄ **JSON** `{"app": {"name": "orders", "log_level": "info"}}`

⇄ **YAML** `app: {name: orders, log_level: info}`

⇄ **INI** `[app]` / `name = orders` / `log_level = info`

GOTCHA

! TOML gets awkward past three levels of nesting; that's a sign JSON or YAML fits better

! JSON written by people breaks on a trailing comma; JSONC / JSON5 fix that but need a different parser

DECISION TABLE

IF YOU NEED…	PICK
People edit it, flat or 2-level	TOML
Deep data, written by code	JSON
The tool requires it (CI, k8s)	YAML
Comments in JSON-like data	JSONC / JSON5
Secrets, per-deploy values	Env vars / .env
Java app settings	.properties or YAML
Validation against a schema	JSON / YAML + JSON Schema

READS WITHOUT A PACKAGE?

LANGUAGE	JSON	TOML	YAML	INI
Python	✔	✔ 3.11+	✖	✔
Node.js	✔	✖	✖	✖
Java	✖	✖	✖	✖ (.properties ✔)
Go	✔	✖	✖	✖
C#	✔	✖	✖	✖

⚠ COMMON MISTAKES

- Putting secrets in a checked-in config file "for now"
- Inventing nested structure in INI with dotted section names that only your parser understands
- Choosing YAML for a new app's settings and adding a parser dependency plus its security notes

✓ TIPS & BEST PRACTICES

- Prefer the format your runtime reads natively so config never needs a dependency
- Write a schema or a typed config class and validate at start-up
- Document every key next to it in comments (another reason JSON is a poor fit for hand-edited config)

QUICK SUMMARY

- TOML — hand-edited settings
- JSON — machine-written data
- YAML — when the tool says so
- env vars — secrets and per-deploy values

TOML Basics

key = value · bare / quoted / dotted keys · strings · integers / floats · booleans · comments # · UTF-8

OBVIOUS KEYS, EXPLICIT TYPES

TOML (Tom's Obvious Minimal Language) is `key = value` pairs grouped into **tables**. Every value has a clear type: strings are always quoted, numbers and booleans never are. There is exactly one way to read each line, which avoids YAML-style guessing. Files must be valid **UTF-8**; comments start with **#**.

NOTES

- Keys are case-sensitive
- Bare keys: `A-Z a-z 0-9 _ -`; anything else needs quotes
- A key can be defined only **once** per table
- Current spec: **TOML 1.1.0** (Dec 2025); most parsers read 1.0

PARSE IT

- **Python** `tomllib.load(open("x.toml", "rb"))` (3.11+, read-only)
- **JavaScript** `import { parse } from "smol-toml"`
- **Java** `new TomlMapper().readTree(file)` (Jackson)

VALUES AND KEYS

```toml
title = "Orders API"          # string (always quoted)
port = 8080                   # integer
ratio = 0.75                  # float
debug = false                 # boolean (lowercase only)
big = 1_000_000               # underscores for readability
mask = 0xFF                   # hex   (also 0o755, 0b1010)
nan_ok = nan                  # also inf, -inf

"key with spaces" = 1         # quoted key
server.host = "0.0.0.0"       # dotted key → table "server"
server.port = 8080

# empty values don't exist: omit the key instead
# port =        ← error
```

CODE EXPLAINED

→ Strings need quotes; `title = Orders API` is an error, not a string

→ Dotted keys create nested tables, same as writing a `[server]` table

→ There is no `null`; leave the key out to mean "not set"

→ Numbers keep their type: `8080` is an integer, `8080.0` is a float

SAME DATA IN…

⇄ **JSON** `{"title": "Orders API", "port": 8080, "server": {"host": "0.0.0.0"}}`

⇄ **YAML** `title: Orders API` · `port: 8080` · `server: {host: 0.0.0.0}`

GOTCHA

! Defining `port` twice, even with the same value, is a parse error

! `True` and `FALSE` are errors; booleans are lowercase `true` / `false` only

VALUE TYPES

TYPE	EXAMPLE
String	"text", 'literal'
Integer	42, -7, 1_000, 0xFF
Float	3.14, 6.02e23, inf, nan
Boolean	true / false
Date-time	2026-11-03T08:00:00Z
Array	[1, 2, 3]
Inline table	{ x = 1, y = 2 }

KEYS

FORM	EXAMPLE
Bare	port, max-retries, v2_api
Quoted	"127.0.0.1" = "local"
Dotted	db.pool.size = 10
Quoted part	site."google.com" = true

⚠ COMMON MISTAKES

- Leaving strings unquoted because INI and YAML allow it
- Writing `key = null`; TOML has no null value
- Saving the file in a non-UTF-8 encoding from an old editor

✓ TIPS & BEST PRACTICES

- Use underscores in big numbers (`timeout_ms = 30_000`) to make them readable
- Prefer dotted keys for one or two settings and `[table]` headers for groups
- Run `taplo fmt` or your editor's TOML formatter to keep spacing consistent

QUICK SUMMARY

- `key = value` — one per line
- strings quoted, numbers / booleans not
- dotted keys — nested tables
- no null — omit the key

Sheet 04

TOML Tables & Inline Tables

[table] · dotted-key tables · [a.b.c] · inline { } tables · multi-line inline tables (1.1) · defining a table once

HEADERS GROUP THE KEYS BELOW THEM

A **table** header [name] starts a group; every key until the next header belongs to it. Nest with dots: [tool.ruff.lint]. An **inline table** { key = value, … } writes a small table on one line. TOML 1.1 lets inline tables span **several lines** with a trailing comma, which used to be the format's biggest annoyance.

NOTES

- [a.b.c] creates a and a.b automatically if they don't exist
- Whitespace and indentation inside tables don't matter
- An inline table is complete; you can't add keys to it later
- A table can be defined only once

PYPROJECT-STYLE TABLES

```toml
[project]
name = "orders"
version = "1.4.0"

[tool.ruff]
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "I"]

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["S101"]

# inline table (one line in TOML 1.0)
point = { x = 1, y = 2 }

# TOML 1.1: multi-line inline table, trailing comma OK
author = {
  name = "Bob",
  email = "bob@example.com",
}
```

CODE EXPLAINED

- → [tool.ruff.lint] is the table tool → ruff → lint
- → Keys under a header belong to it until the next header, regardless of indentation
- → Quoted keys like "tests/*" hold paths and globs
- → Multi-line inline tables only parse with TOML 1.1 parsers; keep them on one line for 1.0 tools

SAME DATA IN…

⇄ **JSON** {"tool": {"ruff": {"line-length": 100, "lint": {"select": ["E", "F", "I"]}}}}

⇄ **YAML** tool: / ruff: / line-length: 100 / lint: {select: [E, F, I]}

GOTCHA

- ! [tool.ruff] twice in one file is an error; merge the keys under one header
- ! After point = { x = 1 }, writing [point] or point.z = 3 is an error

THREE WAYS TO WRITE A TABLE

FORM	EXAMPLE
Header	[server] host = "x"
Dotted key	server.host = "x"
Inline	server = { host = "x" }
Nested header	[server.tls] enabled = true

TABLE RULES

RULE	RESULT
Same header twice	Error
Header after dotted keys made it	Error
Super-table defined later	Allowed
Keys after an inline table	Error
Empty table [x]	Allowed

⚠ COMMON MISTAKES

- Expecting indentation to show nesting; only headers and dots do
- Adding a key to an inline table further down the file
- Using TOML 1.1 multi-line inline tables in a file a 1.0 parser (pip, older tools) reads

✓ TIPS & BEST PRACTICES

- Order tables from general to specific: [tool.ruff] before [tool.ruff.lint]
- Use inline tables for small, fixed shapes like { x = 1, y = 2 }
- Indent keys under headers if it helps you read; parsers ignore it

QUICK SUMMARY

- [a.b] — nested table header
- { k = v } — inline table
- TOML 1.1 — multi-line inline tables
- each table defined once

TOML Arrays & Arrays of Tables

arrays · mixed-type arrays · trailing commas · `[[array.of.tables]]` · nested arrays of tables · when to use which

LISTS OF VALUES AND LISTS OF RECORDS

An **array** is `[ … ]` of values, and can span lines with a trailing comma. A list of **records** uses an **array of tables**: each `[[name]]` header adds a new table to the list `name`. This is how `Cargo.toml` lists binaries, how `pyproject.toml` lists tool overrides, and how you'd list servers or users.

NOTES

- Arrays may mix types since TOML 1.0 (`[1, "a"]`)
- Trailing commas are allowed in arrays
- Each `[[x]]` appends; the order is kept
- Sub-tables of the last element use `[x.sub]` right after it

ARRAYS AND [[ARRAYS OF TABLES]]

```toml
ports = [8080, 8443]
hosts = [
  "alpha",
  "beta",          # trailing comma OK
]
matrix = [[1, 2], [3, 4]]

[[servers]]
name = "web-1"
ip = "10.0.0.1"
roles = ["web"]

[[servers]]
name = "db-1"
ip = "10.0.0.2"

[servers.backup]            # belongs to db-1 (last element)
schedule = "daily"

# short records can be inline instead
users = [ { name = "ann", admin = true }, { name = "bo" } ]
```

CODE EXPLAINED

- → Two `[[servers]]` headers make an array with two tables
- → `[servers.backup]` attaches to the most recent `[[servers]]` entry
- → The inline form fits short records; headers read better for long ones
- → Arrays of arrays (`matrix`) are fine

SAME DATA IN…

⇄ **JSON** `{"servers": [{"name": "web-1", "ip": "10.0.0.1"}, {"name": "db-1", "backup": {"schedule": "daily"}}]}`

⇄ **YAML** `servers:` / `- name: web-1` / `- name: db-1` / `backup: {schedule: daily}`

GOTCHA

- ! Writing `[servers]` (single brackets) for the second server is an error: a table can't be redefined
- ! Once `users` is an inline array, `[[users]]` later in the file is an error

WHICH TO USE

DATA	WRITE IT AS
List of plain values	ports = [80, 443]
Few short records	Inline array of inline tables
Records with many keys	[[records]] headers
Records with sub-tables	[[records]] + [records.sub]

REAL EXAMPLES

FILE	ARRAY OF TABLES
Cargo.toml	[[bin]], [[test]]
pyproject.toml	[[tool.mypy.overrides]]
netlify.toml	[[redirects]]
Hugo config	[[menu.main]]

⚠ COMMON MISTAKES

- Using `[x]` instead of `[[x]]` for a list item
- Putting a sub-table header after a different array's entries, so it attaches to the wrong element
- Very long inline arrays of tables that should have been `[[…]]` blocks

✓ TIPS & BEST PRACTICES

- One array element per `[[ ]]` block, with a blank line between, reads like a list of cards
- Break long arrays across lines with a trailing comma; diffs stay clean
- Convert to JSON (`yq -p toml -o json`) when unsure how a file nests

QUICK SUMMARY

- `[a, b]` — array
- `[[x]]` — append a table to array `x`
- `[x.sub]` — sub-table of the last element
- trailing commas — allowed in arrays

Sheet 06

TOML Strings, Dates & Versions

basic vs literal strings · multi-line """ / ''' · escapes (\e, \xHH in 1.1) · offset / local date-time · TOML 1.0 vs 1.1

FOUR STRING KINDS AND REAL DATES

TOML has **basic** strings ("…", with escapes), **literal** strings ('…', no escapes: good for Windows paths and regex), and multi-line versions of each ("""…""", '''…'''). Dates and times are a **native type**, not strings: offset date-times, local date-times, local dates and local times. TOML 1.1 added \e and \xHH escapes and made seconds optional in times.

NOTES

- A newline right after `"""` is trimmed
- In multi-line basic strings, `\` at line end joins lines
- Literal strings can't contain their own quote; use the other kind
- Dates follow RFC 3339 (`T` or a space between date and time)

STRINGS AND DATES

```toml
greeting = "Hello\tWorld\n"          # basic: escapes work
path = 'C:\Users\bob\config'        # literal: backslashes as-is
regex = '\d{3}-\d{4}'               # literal: no double escaping
quote = "She said \"hi\""
esc = "\e[31mred\e[0m"              # TOML 1.1: \e = ESC
byte = "\x41"                       # TOML 1.1: \xHH = "A"

poem = """
Roses are red
Violets are blue"""
oneline = """\
  joined \
  together"""                       # "joined together"

created = 2026-11-03T08:30:00-06:00 # offset date-time
local   = 2026-11-03T08:30:00       # local date-time
day     = 2026-11-03                # local date
alarm   = 07:30:00                  # local time (1.1: 07:30)
```

CODE EXPLAINED

- → Literal strings are the easy way to write Windows paths and regular expressions
- → The trailing `\` in a multi-line basic string removes the newline and leading spaces after it
- → Parsers return date-times as real date objects (`datetime` in Python)
- → Use the offset form for anything that must mean one moment worldwide

SAME DATA IN…

- ⇄ **JSON** dates are just strings: `"created": "2026-11-03T08:30:00-06:00"`
- ⇄ **YAML** `created: 2026-11-03T08:30:00-06:00` (PyYAML turns it into a datetime too)

GOTCHA

- ! `"C:\Users\new"` in a basic string: `\U` and `\n` are escapes, so this is an error or a newline
- ! `\e` , `\xHH` and times without seconds fail in TOML 1.0 parsers such as `tomllib`

STRING KINDS

KIND	SYNTAX	ESCAPES
Basic	"…"	Yes
Multi-line basic	"""…"""	Yes + line-ending \
Literal	'…'	No
Multi-line literal	'''…'''	No

ESCAPES

ESCAPE	MEANS
\n \t \r	Newline, tab, carriage return
\" \\	Quote, backslash
\uXXXX / \UXXXXXXXX	Unicode code point
\e	Escape char (1.1)
\xHH	Code point < 256 (1.1)

1.0 → 1.1 CHANGES

CHANGE	EXAMPLE
Multi-line inline tables	a = {\n x = 1,\n}
Trailing comma in inline tables	{ x = 1, }
Optional seconds	07:30, 2026-11-03T07:30
New escapes	\e, \xHH

⚠ COMMON MISTAKES

- Writing Windows paths in double quotes and getting escape errors
- Storing dates as strings (`"2026-11-03"`) and losing the native type
- Using TOML 1.1 features in files read by 1.0-only tools

✓ TIPS & BEST PRACTICES

- Single quotes for paths and regex, double quotes when you need escapes
- Add the UTC offset to timestamps that leave your machine
- Check which TOML version your parser supports before using 1.1 syntax

QUICK SUMMARY

- `"…"` — escapes · `'…'` — literal
- `"""` / `'''` — multi-line
- dates — native type, RFC 3339
- TOML 1.1 — `\e` , `\xHH` , optional seconds

INI Files

[section] · key = value / key: value · ; and # comments · no standard spec · interpolation · setup.cfg · .gitconfig

SECTIONS AND STRINGS, NO STANDARD

An **INI** file is `[section]` headers followed by `key = value` lines. It is old, simple and **unstandardized**: each parser decides whether `:` works as a separator, which comment characters count, whether keys are case-sensitive and whether values can span lines. Everything is a string. You'll meet it in `setup.cfg`, `tox.ini`, `.gitconfig`, `php.ini`, `.editorconfig`, `my.cnf` and Windows apps.

NOTES

- Python's `configparser` lowercases keys by default
- Values are strings; `getint()`, `getboolean()` convert them
- `[DEFAULT]` (configparser) provides fallbacks for every section
- No nesting; `[a.b]` is just a section named "a.b" to most parsers

PARSE IT

- **Python** `configparser.ConfigParser().read("app.ini")`
- **JavaScript** `import ini from "ini"; ini.parse(text)`
- **Java** Apache Commons `INIConfiguration` (or `ini4j`)

APP.INI

```
; comment (also # in most parsers)
[DEFAULT]
log_level = info

[server]
host = 0.0.0.0
port = 8080
debug: false              ; ":" works in configparser

[database]
url = postgres://db:5432/orders
pool = 10
dsn = %(url)s?pool=%(pool)s   ; configparser interpolation

[paths]
data = /var/lib/orders
cache = ${paths:data}/cache   ; ExtendedInterpolation syntax
```

CODE EXPLAINED

→ `[DEFAULT]` keys appear in every section unless overridden

→ `%(name)s` is configparser's **basic interpolation**; `${section:key}` needs `ExtendedInterpolation`

→ `debug: false` reads as the string `"false"`; `getboolean` turns it into `False`

→ Inline `;` comments only work in some parsers (configparser needs `inline_comment_prefixes`)

SAME DATA IN…

⇄ **TOML** `[server]` / `port = 8080` / `debug = false` (typed)

⇄ **.properties** `server.port=8080`

⇄ **.env** `SERVER_PORT=8080`

GOTCHA

! A literal `%` in a configparser value must be `%%`, or interpolation raises an error

! `.gitconfig` is INI-like but has its own rules (subsections like `[remote "origin"]`)

PARSER DIFFERENCES

FEATURE	CONFIGPARSER	NODE INI	WINDOWS
Separators	= and :	=	=
Comments	# ;	; #	;
Case-sensitive keys	No (lowercased)	Yes	No
Duplicate keys	Error	Last wins / arrays	First wins
Multi-line values	Indented lines	No	No

INI FILES YOU'LL MEET

FILE	TOOL
setup.cfg / tox.ini	Python packaging, tox
.gitconfig	Git
.editorconfig	EditorConfig
php.ini	PHP
my.cnf	MySQL
desktop.ini	Windows Explorer

⚠ COMMON MISTAKES

- Assuming INI syntax is the same everywhere and moving a file between tools
- Reading `port` and comparing it to an integer without converting it
- Writing a `%` in a password inside a configparser file

✓ TIPS & BEST PRACTICES

- Use `configparser.RawConfigParser` or `interpolation=None` when values contain `%`
- Set `optionxform = str` on configparser to keep key case
- For new Python projects, put tool config in `pyproject.toml` instead of `setup.cfg`

QUICK SUMMARY

- `[section]` + `key = value`
- no spec — parser decides the details
- everything is a string
- `[DEFAULT]` + interpolation — configparser extras

Sheet 08

Java .properties Files

key=value · : and space separators · \ line continuation · escapes · encoding (UTF-8 vs ISO-8859-1) · Spring `application.properties`

FLAT KEYS FOR THE JVM

A **.properties** file is the standard Java way to store settings and translations: one `key=value` per line, flat, all strings. Nesting is faked with **dotted keys** (`spring.datasource.url`). The separator can be `=`, `:` or whitespace; comments start with `#` or `!`. Spring Boot, Gradle (`gradle.properties`), Maven filtering and Java `ResourceBundle` i18n all use it.

NOTES

- Leading whitespace before keys and values is ignored
- A `\` at line end continues the value on the next line
- Lists are a convention: `a.b[0]=x` (Spring) or comma-separated values
- The Java set covers `java.util.Properties` in code

PARSE IT

- **Python** `jproperties`: `Properties().load(f, "utf-8")`
- **JavaScript** `properties-reader("app.properties").get("server.port")`
- **Java** `props.load(Files.newBufferedReader(path))` (UTF-8)

APPLICATION.PROPERTIES

```
# Server
server.port=8080
server.servlet.context-path=/api

! "!" starts a comment too
spring.datasource.url=jdbc:postgresql://db:5432/orders
spring.datasource.username=${DB_USER:app}
spring.datasource.password=${DB_PASSWORD}

# Spaces around "=" are trimmed
app.greeting = Hello, World
# Oops: everything after "=" is the value, "# dev" included
app.mode=fast # dev
app.cors.allowed-origins=https://a.example,https://b.example
app.servers[0]=web-1
app.servers[1]=web-2
app.long-text=This value goes on \
              across two lines
# Backslashes must be doubled
app.path=C:\\data\\orders
```

CODE EXPLAINED

- → `${DB_USER:app}` is Spring placeholder syntax with a default; plain `Properties` doesn't expand it
- → Comments must start the line: `# …` after a value becomes part of the value
- → Spring binds `app.servers[0]` and comma lists into `List<String>` fields
- → A continued line drops the leading spaces on the next line

SAME DATA IN…

- ⇄ **YAML** (`application.yml`) `spring:` / `datasource:` / `url: jdbc:postgresql://db:5432/orders`
- ⇄ **.env** `SPRING_DATASOURCE_URL=jdbc:…` (Spring's relaxed binding maps it)
- ⇄ **TOML** `[spring.datasource]` / `url = "jdbc:…"`

GOTCHA

- ! `Properties.load(InputStream)` reads **ISO-8859-1**; non-Latin text needs `load(Reader)` with UTF-8 or `\uXXXX` escapes
- ! There are no inline comments; `port=8080 # dev` sets port to `"8080 # dev"`

SYNTAX RULES

WRITE	MEANS
`key=value`	Standard form
`key: value / key value`	Also valid separators
`# or ! at line start`	Comment
`\ at line end`	Continue value
`\=, \:, \ (space)`	Literal separator in a key
`\uXXXX`	Unicode character

WHERE YOU'LL SEE IT

FILE	READ BY
`application.properties`	Spring Boot
`application-dev.properties`	Spring profile "dev"
`gradle.properties`	Gradle (+ ~/.gradle)
`messages_fr.properties`	ResourceBundle i18n
`log4j2.properties`	Log4j 2

⚠ COMMON MISTAKES

- Adding `# comment` after a value on the same line
- Single backslashes in Windows paths; `\d` becomes `d`
- Saving non-ASCII text and reading it with the `InputStream` overload

✓ TIPS & BEST PRACTICES

- Keep one profile file per environment (`application-prod.properties`) and secrets in env vars
- Sort keys and group them with blank lines and comments; the file has no structure otherwise
- Prefer `application.yml` in Spring when the keys get deep and repetitive

QUICK SUMMARY

- `key=value` — flat strings
- dotted keys — fake nesting
- `\` — continuation and escapes
- encoding — use a UTF-8 `Reader`

.env Files

KEY=value · quoting · export prefix · variable expansion · .env.example · never commit secrets · loaders per stack

ENVIRONMENT VARIABLES IN A FILE

A **.env** file lists KEY=value lines that a loader copies into the process environment at start-up, so local development can mimic production without exporting variables by hand. It is a **convention**, not a standard: the dotenv libraries, Docker Compose, Node's `--env-file` and shells each parse it slightly differently. It almost always holds secrets, so it stays **out of Git**; commit a `.env.example` instead.

NOTES

- Loaders usually **don't override** variables already set in the real environment
- Values are strings; your app converts them
- Docker Compose reads `.env` for `${VAR}` interpolation in `compose.yaml`
- Many frameworks load `.env.local`, `.env.development` etc. in order

PARSE IT

- **Python** `from dotenv import load_dotenv; load_dotenv()`
- **JavaScript** `node --env-file=.env app.js` or `process.loadEnvFile()`
- **Java** `Dotenv.load().get("DB_URL")` (dotenv-java)

.ENV AND .ENV.EXAMPLE

```
# .env  (gitignored)
APP_ENV=development
PORT=3000
DATABASE_URL=postgres://app:s3cret@localhost:5432/orders
API_KEY="sk_test_51H…"          # quotes are stripped
GREETING='Hello, $USER'          # single quotes: no expansion
MULTILINE="line one\nline two"   # \n expanded in double quotes (most loaders)
LOG_DIR=${HOME}/logs             # expansion: supported by some loaders
export DEBUG=false               # "export" prefix allowed by most loaders

# .env.example  (committed: names, no real values)
APP_ENV=development
PORT=3000
DATABASE_URL=
API_KEY=

# load into a shell
set -a; source .env; set +a
```

CODE EXPLAINED

→ The loader sets `process.env.PORT` / `os.environ["PORT"]` to the **string** `"3000"`

→ `${HOME}` expansion and `\n` handling vary by loader; test before relying on them

→ `set -a` exports everything a `source` defines, so child processes see it

→ `.env.example` documents every variable the app needs

SAME DATA IN…

⇄ **.properties** `database.url=postgres://…`

⇄ **YAML** (Compose) `environment:` / `DATABASE_URL: postgres://…`

⇄ **Shell** `export DATABASE_URL=postgres://…`

GOTCHA

! A real environment variable wins over `.env` with most loaders, so a stale `export` in your shell silently overrides the file

! Spaces around `=` (`PORT = 3000`) break some loaders and the shell

LOADERS

STACK	LOADER
Node.js 20.6+	node --env-file=.env
Node (older)	dotenv package
Python	python-dotenv
Vite / Next.js	Built in (.env, .env.local, .env.[mode])
Docker Compose	.env + env_file:
Spring Boot	spring.config.import=optional:file:.env[.properties]
Shell	set -a; source .env; set +a

SAFE PRACTICE

DO	WHY
.env in .gitignore	Secrets never reach the repo
Commit .env.example	Lists required keys
Validate at start-up	Fail fast on a missing key
Real env in production	.env is for local dev
Rotate leaked keys	Deleting the commit isn't enough

⚠ COMMON MISTAKES

- Committing `.env` once; the secret stays in Git history even after deleting the file
- Expecting `.env` changes to apply without restarting the process
- Using `PORT` as a number without converting the string

✓ TIPS & BEST PRACTICES

- Add `.env` to `.gitignore` before creating it
- Validate env vars into a typed config object at start-up (Zod, Pydantic Settings)
- Use a secrets manager or the platform's env settings in production

QUICK SUMMARY

- `KEY=value` — copied into the environment
- strings only — convert in code
- `.env.example` — commit names, not values
- loaders differ — expansion, quotes, overrides

Environment Variables & Config Layering

LATER LAYERS OVERRIDE EARLIER ONES

Real apps merge config from **layers**: built-in defaults, then a checked-in file, then a per-environment file, then **environment variables**, then **command-line flags**. Each layer overrides the one before, so the same build runs everywhere with different settings. The **12-factor** rule: anything that changes between deploys (URLs, credentials, feature flags) comes from the environment, not from code.

NOTES

- Env var names: `UPPER_SNAKE_CASE`, letters, digits, `_`
- Nesting by convention: `APP_DB__HOST` (double underscore) in .NET, Pydantic, many libraries
- `NODE_ENV`, Spring profiles, `ASPNETCORE_ENVIRONMENT` pick the per-env file
- Kubernetes ConfigMaps / Secrets and CI secrets arrive as env vars

LAYERS IN PRACTICE (SPRING BOOT ORDER)

```
# 1. defaults in code       @Value("${server.port:8080}")
# 2. application.yml        server: { port: 8080 }
# 3. application-prod.yml   server: { port: 80 }      (profile "prod")
# 4. environment variable   SERVER_PORT=9000
# 5. command line           java -jar app.jar --server.port=9100
#    → final value: 9100

# Same idea in other stacks
# Node:    defaults → config/default.json → config/production.json
#          → process.env → argv           (the "config" package)
# .NET:    appsettings.json → appsettings.Production.json
#          → env vars (Logging__LogLevel__Default) → args
# Python:  pydantic-settings: defaults → .env → env vars → init args
```

CODE EXPLAINED

→ Spring maps `SERVER_PORT` to `server.port` (relaxed binding: uppercase, `.` → `_`)

→ In .NET, `__` in an env var name means a nested key (`Logging:LogLevel:Default`)

→ Command-line flags are the last word, good for one-off overrides

→ Log the final merged config (minus secrets) at start-up to debug layering

SAME DATA IN…

⇄ **.env** `SERVER_PORT=9000`

⇄ **TOML** `[server]` / `port = 9000`

⇄ **CLI** `--server.port=9000` · `PORT=9000 npm start`

GOTCHA

! A variable set in CI or the container image overrides your file and is easy to forget

! Env vars can't hold structure; complex values arrive as JSON strings or comma lists you must parse

TYPICAL PRECEDENCE (LOW → HIGH)

LAYER	HOLDS
Defaults in code	Safe fallbacks
Base config file	Shared settings, committed
Env-specific file	dev / staging / prod differences
Local overrides	.env, *.local files, gitignored
Environment variables	Secrets, per-deploy values
CLI flags	One-off overrides

ENV VAR NAMING

STACK	NESTED KEY AS ENV VAR
Spring Boot	SPRING_DATASOURCE_URL
.NET	ConnectionStrings__Default
Pydantic Settings	APP_DB__HOST (env_nested_delimiter)
Vite (exposed to browser)	VITE_API_URL
Next.js (browser)	NEXT_PUBLIC_API_URL
Docker Compose	${VAR:-default}

⚠ COMMON MISTAKES

- Reading `process.env` all over the code instead of once into a config object
- Shipping browser-exposed vars (`VITE_`, `NEXT_PUBLIC_`) that contain secrets
- Not knowing which layer won, then "fixing" the wrong file

✓ TIPS & BEST PRACTICES

- Load and validate config in one module; the rest of the app imports typed values
- Fail start-up when a required variable is missing
- Keep secrets in the platform (Kubernetes Secrets, GitHub secrets, a vault), injected as env vars

QUICK SUMMARY

- layers — defaults → files → env → flags
- 12-factor — deploy-specific values from env
- `__` — nesting in env var names
- validate once — typed config object

JSON for Config (JSONC & JSON5)

no comments in JSON · JSONC (`tsconfig`, VS Code) · JSON5 · trailing commas · `$schema` key · when JSON is the wrong choice

STRICT JSON AND ITS FRIENDLIER COUSINS

Plain **JSON** has no comments and no trailing commas, which is fine for files programs write and painful for files people edit. Two relaxed variants exist: **JSONC** (JSON with comments, used by VS Code settings and `tsconfig.json`) and **JSON5** (comments, trailing commas, unquoted keys, single quotes). Whether a file accepts them depends entirely on the tool reading it.

NOTES

- `package.json` is strict JSON: a comment breaks npm
- A `"$schema"` key points editors at a JSON Schema for autocomplete and checks
- Babel (`babel.config.json5`), Renovate and others accept JSON5
- The JSON set covers syntax, schema and parsing in depth

PARSE IT

- **Python** `json5.loads(text)` or `commentjson`
- **JavaScript** `JSON5.parse(text)` or Microsoft `jsonc-parser`
- **Java** Jackson `JsonReadFeature.ALLOW_JAVA_COMMENTS`

JSON, JSONC AND JSON5 SIDE BY SIDE

```
// tsconfig.json — JSONC: comments OK
{
  "$schema": "https://json.schemastore.org/tsconfig",
  "compilerOptions": {
    "strict": true,      // keep this on
    "target": "es2022",  // trailing comma below is OK in tsconfig
  },
}

// renovate.json5 — JSON5
{
  extends: ['config:recommended'],   // unquoted keys, single quotes
  schedule: ['before 6am on monday'],
  automerge: true,
}

// package.json — strict JSON: none of the above allowed
```

CODE EXPLAINED

→ TypeScript and VS Code parse their JSON files as JSONC, so comments and trailing commas work

→ JSON5 also allows hex numbers, `Infinity`, `NaN` and multi-line strings

→ `"$schema"` is ignored by the tool but used by editors for validation

→ Rename to `.jsonc` / `.json5` where the tool allows, so editors highlight it correctly

SAME DATA IN…

⇄ **TOML** `[compilerOptions]` / `strict = true # keep this on`

⇄ **YAML** `compilerOptions:` / `strict: true # keep this on`

GOTCHA

! `JSON.parse` on a `tsconfig.json` with comments throws; read it with the TypeScript API or a JSONC parser

! Some tools accept comments but **rewrite** the file and drop them (e.g. `npm pkg set`)

WHAT EACH ALLOWS

FEATURE	JSON	JSONC	JSON5
`Comments // /* */`	✖	✔	✔
`Trailing commas`	✖	Usually	✔
`Unquoted keys`	✖	✖	✔
`Single quotes`	✖	✖	✔
`Hex, Infinity, NaN`	✖	✖	✔

WHICH FILES ALLOW WHAT

FILE	FORMAT
`package.json`	JSON
`tsconfig.json, jsconfig.json`	JSONC
`.vscode/*.json`	JSONC
`.eslintrc.json (legacy)`	JSONC
`renovate.json5, babel.config.json5`	JSON5
`composer.json, appsettings.json`	JSON (appsettings allows comments)

⚠ COMMON MISTAKES

- Adding a comment to `package.json` and breaking every npm command
- Reading a JSONC file with a strict parser in a build script
- Hand-editing a big JSON config when the tool also supports a JS or TOML config

✓ TIPS & BEST PRACTICES

- Add `"$schema"` to JSON config files for free autocomplete in VS Code
- Use a `"//"` key for notes in strict JSON if you must: `"//": "why this is pinned"`
- Prefer the tool's JS / TS config (`eslint.config.js`, `vite.config.ts`) when you need logic or comments

QUICK SUMMARY

- JSON — strict, no comments
- JSONC — comments (+ trailing commas in TS / VS Code)
- JSON5 — comments, unquoted keys, single quotes
- `"$schema"` — editor validation

YAML for Config — Traps

implicit typing (`NO`, `on`, `010`, `1e3`) · YAML 1.1 vs 1.2 · quoting rules · indentation · anchors for DRY config · safe loading

READABLE UNTIL IT GUESSES WRONG

YAML guesses types from how a value looks. In **YAML 1.1** (PyYAML, SnakeYAML, many older tools), `yes`, `no`, `on`, `off` are booleans, `010` is octal and `22:30` is a base-60 number. **YAML 1.2** fixed most of this, but your tool may not use it. The safe habit: **quote anything that isn't obviously a number or `true` / `false`**. This page is the config-file checklist; the YAML set covers the full language.

NOTES

- Indentation must be spaces, never tabs
- Use `yaml.safe_load`, never `yaml.load` on untrusted input
- Anchors `&` / aliases `*` / merge `<<:` remove repetition
- GitHub Actions treats `on:` as a key because its parser is not YAML 1.1 strict

PARSE IT

- **Python** `yaml.safe_load(f)` (PyYAML, YAML 1.1) · `ruamel.yaml` (1.2, keeps comments)
- **JavaScript** `import YAML from "yaml"; YAML.parse(text)` (1.2)
- **Java** SnakeYAML `new Yaml(new SafeConstructor(new LoaderOptions()))`

VALUES YAML CHANGES BEHIND YOUR BACK

```yaml
country: NO           # 1.1: false   → write "NO"
enabled: on           # 1.1: true    → write "on" or true
version: 1.10         # float 1.1    → write "1.10"
zip: 01234            # 1.1: octal 668 → write "01234"
time: 22:30           # 1.1: 1350 (base 60) → write "22:30"
size: 1e3             # 1.2: float 1000.0; 1.1: string
id: 0x1F              # 31
empty:                # null, not ""
tilde: ~              # null
token: ${API_TOKEN}   # just a string; YAML has no env expansion

# DRY with anchors and merge keys
defaults: &defaults
  retries: 3
  timeout: 30
service_a:
  <<: *defaults
  timeout: 60         # override one key
```

CODE EXPLAINED

- → Every "fix" is the same: quote it
- → `version: 1.10` loses the trailing zero because it becomes the float `1.1`
- → An empty value is `null`; write `""` for an empty string
- → `${API_TOKEN}` only works if the tool (Compose, Actions) expands it; plain YAML doesn't

SAME DATA IN…

- ⇄ **TOML** `country = "NO"` · `version = "1.10"` · `time = 22:30:00` (typed, no guessing)
- ⇄ **JSON** `{"country": "NO", "version": "1.10"}`

GOTCHA

- ! `yaml.load(text)` without a safe loader can construct arbitrary Python objects from tagged input
- ! Merge keys `<<:` are YAML 1.1; some 1.2-only parsers reject or ignore them

YAML 1.1 VS 1.2

VALUE	YAML 1.1	YAML 1.2 CORE
`yes / no / on / off`	Boolean	String
`True / FALSE`	Boolean	Boolean
`010`	8 (octal)	10
`0o10`	String	8
`22:30`	1350	String
`1e3`	String	1000.0

WHICH YAML YOUR TOOL SPEAKS

TOOL / LIBRARY	VERSION
`PyYAML`	1.1
`ruamel.yaml`	1.2 (default)
`js-yaml 4 / yaml (eemeli)`	1.2
`SnakeYAML`	1.1
`snakeyaml-engine`	1.2
`Go yaml.v3`	Mostly 1.2

⚠ COMMON MISTAKES

- Unquoted country codes, versions, ZIP codes and times
- Tabs for indentation, copied from another file
- Assuming `${VAR}` expands in every YAML file

✓ TIPS & BEST PRACTICES

- Quote every string that could look like a number, boolean or date
- Run `yamllint` in CI with `truthy` and `octal-values` rules on
- Use a schema (sheet 19) so the editor flags wrong types as you type

QUICK SUMMARY

- quote it — the fix for most traps
- 1.1 vs 1.2 — know your parser
- `safe_load` — never `load`
- `&` / `*` / `<<:` — DRY config

package.json

THE MANIFEST OF EVERY NODE PROJECT

`package.json` describes a JavaScript package: its name and version, the **scripts** you run with `npm run`, the **dependencies** it installs, and how other code imports it (`exports`, `type`). It is strict JSON. npm, pnpm, Yarn and Bun all read it; the exact installed versions live in the **lock file** next to it.

NOTES

- `npm pkg get version` / `npm pkg set scripts.lint="eslint ."` edit it safely
- `"type": "module"` makes `.js` files ES modules
- `devDependencies` are only for building and testing
- The Node set covers npm commands in depth

PACKAGE.JSON

```
{
  "name": "@acme/orders",
  "version": "1.4.0",
  "type": "module",
  "private": true,
  "engines": { "node": ">=22" },
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "test": "vitest run",
    "lint": "eslint ."
  },
  "dependencies": {
    "express": "^5.1.0",
    "zod": "~4.1.0"
  },
  "devDependencies": {
    "typescript": "5.9.3",
    "vitest": "^5.1.0"
  },
  "exports": { ".": "./dist/index.js" }
}
```

CODE EXPLAINED

→ `^5.1.0` allows 5.x updates (≥5.1.0 <6.0.0); `~4.1.0` allows patches only (<4.2.0)

→ An exact version (`5.9.3`) never moves; the lock file pins the rest

→ `"private": true` stops accidental `npm publish`

→ `exports` controls what `import "@acme/orders"` resolves to

SAME DATA IN…

⇄ **pyproject.toml** `[project]` / `dependencies = ["fastapi>=0.115"]` / `[project.scripts]`

⇄ **Maven** `<dependency>` blocks · **Gradle** `implementation("…")`

GOTCHA

! `^0.2.3` only allows `0.2.x` : caret ranges are stricter below 1.0.0

! Scripts run with `node_modules/.bin` on `PATH` , so `vitest` works there but not in your shell

SEMVER RANGES

RANGE	ALLOWS
`^1.2.3`	>=1.2.3 <2.0.0
`~1.2.3`	>=1.2.3 <1.3.0
`1.2.3`	Exactly 1.2.3
`>=1.2 <2`	Explicit range
`*`	Anything (avoid)
`workspace:*`	Local workspace package (pnpm / Yarn)

FIELDS WORTH KNOWING

FIELD	PURPOSE
`scripts`	npm run <name>
`bin`	CLI commands this package installs
`files`	What gets published
`peerDependencies`	Host must supply it (plugins)
`overrides`	Force a nested dependency version
`packageManager`	Pin pnpm / Yarn via Corepack

⚠ COMMON MISTAKES

- Putting build tools in `dependencies` , so they ship to production
- Deleting the lock file to "fix" an install and getting different versions everywhere
- Adding comments; strict JSON fails to parse

✓ TIPS & BEST PRACTICES

- Use `npm pkg set` in scripts instead of hand-editing with `sed`
- Pin the Node version in `engines` and `.nvmrc` (sheet 18)
- Use `npm ci` in CI so the lock file is followed exactly

QUICK SUMMARY

- `scripts` — `npm run <name>`
- `^` — minor updates · `~` — patch updates
- `devDependencies` — build / test only
- lock file — exact versions

tsconfig.json & jsconfig.json

HOW TYPESCRIPT READS YOUR PROJECT

`tsconfig.json` (JSONC) tells the TypeScript compiler and your editor which files belong to the project and how to check them. **target** sets the JavaScript version emitted, **module** / **moduleResolution** decide how imports work, **strict** turns on the full type checks, and **paths** maps import aliases. `jsconfig.json` is the same file for JavaScript projects, giving VS Code IntelliSense without TypeScript.

NOTES

- Run `npx tsc --showConfig` to see the merged result
- Bundler projects (Vite, Next) use `moduleResolution: "bundler"` and `noEmit`
- Node projects use `module: "nodenext"`
- The JS / TS set covers the type system itself

TSCONFIG.JSON FOR A VITE APP

```jsonc
{
  "extends": "./tsconfig.base.json",
  "compilerOptions": {
    "target": "es2022",
    "lib": ["es2023", "dom", "dom.iterable"],
    "module": "esnext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "verbatimModuleSyntax": true,
    "noEmit": true,
    "skipLibCheck": true,
    "paths": { "@/*": ["./src/*"] },
    "types": ["vitest/globals"]
  },
  "include": ["src", "tests"],
  "exclude": ["dist"]
}
```

CODE EXPLAINED

- → `extends` pulls in a shared base; keys here override it
- → `noEmit` because Vite (esbuild / Rolldown) compiles; `tsc` only type-checks
- → `paths` needs the bundler configured with the same alias (`resolve.alias` in Vite)
- → `types` limits which `@types` packages load globally

SAME DATA IN…

- ⇄ **jsconfig.json** same `compilerOptions` subset with `"checkJs": true` for JS projects
- ⇄ **pyproject.toml** `[tool.mypy]` / `strict = true` — the Python equivalent

GOTCHA

- ! `paths` only affects type checking; without a matching bundler / runtime alias, imports fail at run time
- ! `include` defaults to every `.ts` file under the folder; a stray `dist/` gets type-checked

KEY OPTIONS

OPTION	COMMON VALUE	EFFECT
`target`	es2022	Output JS version
`module`	esnext / nodenext / preserve	Import / export style
`moduleResolution`	bundler / nodenext	How imports resolve
`strict`	true	All strict checks
`noEmit`	true	Type-check only
`outDir / rootDir`	dist / src	Where output goes
`isolatedModules`	true	Safe for per-file compilers

RECIPES

PROJECT	MODULE / RESOLUTION
`Vite / Next / bundler`	esnext / bundler
`Node ESM library`	nodenext / nodenext
`Node CommonJS`	nodenext + "type": "commonjs"
`Monorepo`	"references" + composite

⚠ COMMON MISTAKES

- Copying a tsconfig between a Node app and a Vite app; the module settings differ
- Setting `paths` and forgetting the bundler / test-runner alias
- Turning `strict` off to silence errors instead of fixing types

✓ TIPS & BEST PRACTICES

- Start from the framework's template (`npm create vite`) or `@tsconfig/bases` packages
- Use `tsc --noEmit` in CI as a separate type-check step
- Split `tsconfig.app.json` / `tsconfig.node.json` when browser and Node code live together

QUICK SUMMARY

- `strict` — full type checking
- `module` + `moduleResolution` — match your runtime / bundler
- `paths` — aliases (configure the bundler too)
- `extends` — share a base

pyproject.toml

[project] · dependencies · [build-system] · [project.scripts] · [tool.*] (ruff · pytest · mypy) · dependency groups

ONE FILE FOR A PYTHON PROJECT

`pyproject.toml` is the standard Python project file. `[project]` holds metadata and dependencies (PEP 621); `[build-system]` names the build backend; `[dependency-groups]` (PEP 735) holds dev-only groups; and `[tool.<name>]` tables configure tools such as Ruff, pytest, mypy, coverage and uv. It replaces `setup.py`, `setup.cfg`, `requirements-dev.txt` and most tool-specific config files.

NOTES

- Dependencies use PEP 508 strings: `"httpx>=0.27"`, `"rich[jupyter]"`
- `uv`, `pip`, Poetry 2, Hatch and PDM all read `[project]`
- `license = "MIT"` is an SPDX expression (PEP 639)
- The Python and pytest sets cover the tools themselves

PARSE IT

- **Python** `tomllib.load(open("pyproject.toml", "rb"))["project"]["version"]`
- **JavaScript** `parse(readFileSync("pyproject.toml", "utf8"))` (smol-toml)
- **Java** `Toml.parse(path).getString("project.version")` (tomlj)

PYPROJECT.TOML

```toml
[project]
name = "orders"
version = "1.4.0"
requires-python = ">=3.12"
license = "MIT"
dependencies = [
  "fastapi>=0.115",
  "pydantic-settings>=2.6",
]

[project.optional-dependencies]
postgres = ["psycopg[binary]>=3.2"]

[project.scripts]
orders = "orders.cli:main"      # installs an `orders` command

[dependency-groups]
dev = ["pytest>=9", "ruff>=0.14", "mypy>=1.18"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[tool.ruff]
line-length = 100

[tool.pytest]                   # pytest 9 native TOML
testpaths = ["tests"]
```

CODE EXPLAINED

- → `optional-dependencies` are extras users opt into: `pip install orders[postgres]`
- → `dependency-groups` are for development and never published
- → `[project.scripts]` creates a console command when the package is installed
- → pytest 9 reads `[tool.pytest]`; older versions need `[tool.pytest.ini_options]`

SAME DATA IN…

- ⇄ **package.json** `"dependencies"` · `"devDependencies"` · `"bin"` · `"scripts"`
- ⇄ **Cargo.toml** `[package]` · `[dependencies]` · `[dev-dependencies]`

GOTCHA

- ! Tools that read a missing `[tool.x]` table fall back to their own files or defaults silently
- ! `requires-python` is checked at install time, not when you run the code

TABLES

TABLE	HOLDS
`[project]`	Name, version, deps (PEP 621)
`[project.optional-dependencies]`	Extras
`[dependency-groups]`	Dev groups (PEP 735)
`[project.scripts]`	Console commands
`[build-system]`	Build backend
`[tool.*]`	Tool settings

BUILD BACKENDS

BACKEND	REQUIRES / BUILD-BACKEND
Hatchling	hatchling / hatchling.build
uv	uv_build / uv_build
Setuptools	setuptools>=77 / setuptools.build_meta
Poetry	poetry-core / poetry.core.masonry.api
Flit	flit_core / flit_core.buildapi

⚠ COMMON MISTAKES

- Putting pytest and Ruff in `dependencies`, so they install for users
- Keeping both `setup.py` and `[project]` metadata and letting them drift
- Writing `[tool.pytest]` with pytest 8 or older, which ignores it

✓ TIPS & BEST PRACTICES

- Let `uv add` / `uv add --dev` edit the file and the lock file together
- Move tool configs (`.flake8`, `pytest.ini`, `.coveragerc`) into `[tool.*]` tables
- Use `requires-python` to stop installs on unsupported versions

QUICK SUMMARY

- `[project]` — metadata + dependencies
- `[dependency-groups]` — dev-only deps
- `[build-system]` — how to build
- `[tool.*]` — every tool's settings

compose.yaml (Docker Compose)

A LOCAL STACK IN ONE FILE

`compose.yaml` defines the **services** (containers) of an app and how they connect: which image or Dockerfile each uses, published **ports**, **environment**, **volumes** and start order. `docker compose up` starts it all. Compose reads a `.env` file next to it for `${VAR}` **interpolation**, which is separate from the environment passed into containers.

NOTES

- The top-level `version:` key is obsolete; leave it out
- Services reach each other by service name (`db:5432`)
- `compose.override.yaml` is merged automatically for local tweaks
- The Docker set covers Compose commands in depth

COMPOSE.YAML

```yaml
services:
  api:
    build: .
    ports:
      - "8080:3000"                 # host:container (quote it!)
    env_file: .env                  # into the container
    environment:
      DATABASE_URL: postgres://app:${DB_PASSWORD}@db:5432/orders
      LOG_LEVEL: ${LOG_LEVEL:-info} # interpolated from .env / shell
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD:?set DB_PASSWORD}
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
  adminer:
    image: adminer
    profiles: [tools]               # only with --profile tools
volumes:
  pgdata:
```

CODE EXPLAINED

- → `${VAR:-default}` uses a default; `${VAR:?message}` stops with an error if unset
- → `env_file` passes variables into the container; `.env` interpolation fills in the YAML itself
- → `condition: service_healthy` waits for the healthcheck, not just container start
- → Named volume `pgdata` keeps data between `down` and `up`

SAME DATA IN…

- ⇄ **Kubernetes** `Deployment` + `Service` + `PersistentVolumeClaim` (see the Kubernetes set)
- ⇄ **docker run** `docker run -p 8080:3000 --env-file .env myapi`

GOTCHA

- ! Unquoted `- 22:22` can be read as a base-60 number by YAML 1.1 parsers; always quote port mappings
- ! Changing `.env` needs `docker compose up -d` again; running containers keep old values

SERVICE KEYS

KEY	SETS
`image / build`	What to run
`ports`	Publish to the host
`environment / env_file`	Container env
`volumes`	Named volumes or bind mounts
`depends_on`	Start order (+ health)
`restart`	unless-stopped / always
`profiles`	Optional services

COMMANDS

COMMAND	DOES
`docker compose up -d`	Start in background
`docker compose config`	Print the merged, interpolated file
`docker compose logs -f api`	Follow logs
`docker compose down -v`	Stop and delete volumes
`--profile tools up`	Include optional services

⚠ COMMON MISTAKES

- Putting passwords directly in `compose.yaml` and committing it
- Expecting `depends_on` alone to wait until the database accepts connections
- Using `down -v` and losing the database volume by accident

✓ TIPS & BEST PRACTICES

- Run `docker compose config` to see exactly what Compose will use after interpolation
- Use `${VAR:?message}` for required secrets so a missing one fails loudly
- Keep dev-only services behind `profiles`

QUICK SUMMARY

- `services` — one container each
- `${VAR:-default}` — interpolation from `.env`
- `env_file` — env inside the container
- `depends_on` + healthcheck — real start order

GitHub Actions Workflow Files

CI AS A YAML FILE IN THE REPO

A workflow is a YAML file in `.github/workflows/`. **on** lists the events that start it, **jobs** run in parallel on **runners** (`runs-on`), and each job's **steps** either `run` shell commands or `uses` a reusable action with inputs in `with`. `${{ }}` **expressions** read contexts such as `github`, `matrix`, `secrets` and `env`. This sheet is the file format; the GitHub set covers Actions in depth.

NOTES

- Indentation errors fail the whole workflow; the Actions tab shows the line
- Pin third-party actions to a full commit SHA
- `workflow_dispatch` adds a manual **Run workflow** button
- Validate with `actionlint` before pushing

.GITHUB/WORKFLOWS/CI.YML

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node: [22, 24]
    env:
      CI: true
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v6
        with:
          node-version: ${{ matrix.node }}
          cache: npm
      - run: npm ci
      - run: npm test
        env:
          API_KEY: ${{ secrets.API_KEY }}
```

CODE EXPLAINED

→ `pull_request:` with no value means "all pull request events"

→ `permissions` narrows the default token to read-only

→ The matrix runs the job once per Node version

→ Step-level `env` scopes a secret to the one step that needs it

SAME DATA IN…

⇄ **GitLab CI** (`.gitlab-ci.yml`) `test:` / `image: node:22` / `script: [npm ci, npm test]`

⇄ **Azure Pipelines** `trigger: [main]` / `steps:` / `- script: npm test`

GOTCHA

! `on:` is a plain key to GitHub's parser, but a YAML 1.1 tool (`yamllint` default) may flag it as the boolean `true`

! Untrusted input like `${{ github.event.pull_request.title }}` inside `run:` is a script-injection risk; pass it through `env`

COMMON TRIGGERS

ON:	RUNS WHEN
push	Commits pushed (filter branches / paths)
pull_request	PR opened or updated
workflow_dispatch	Run by hand
schedule	Cron, e.g. - cron: "0 6 * * 1"
release	A release is published
workflow_call	Called by another workflow

EXPRESSION CONTEXTS

CONTEXT	EXAMPLE
github	github.ref_name, github.sha
matrix	matrix.node
secrets	secrets.API_KEY
env / vars	env.CI, vars.REGION
steps	steps.build.outputs.path
needs	needs.build.result

⚠ COMMON MISTAKES

- Tabs or uneven indentation in a workflow file
- Putting secrets in `env` at the workflow level where every step can read them
- Using `@main` for third-party actions instead of a pinned SHA

✓ TIPS & BEST PRACTICES

- Run `actionlint` locally or in pre-commit to catch errors before CI does
- Use the GitHub Actions extension or SchemaStore schema for autocomplete (sheet 19)
- Keep jobs small and use `needs` to chain them

QUICK SUMMARY

- `on` — triggers
- `jobs.<id>.steps` — `run` or `uses`
- `${{ }}` — expressions over contexts
- `permissions` + secrets in step `env` — least privilege

Project Dotfiles

SMALL FILES THAT SET PROJECT RULES

Most repos carry a handful of **dotfiles** at the root. `.editorconfig` sets indentation and line endings for every editor; `.gitignore` and `.gitattributes` tell Git what to skip and how to treat files; `.npmrc` configures npm; `.nvmrc` and `.python-version` pin the runtime version; `.prettierrc` holds formatter rules. Each is tiny, and each saves a class of "works on my machine" problems.

NOTES

- Formats vary: `.editorconfig` is INI-like, `.gitignore` is glob lines, `.prettierrc` is JSON or YAML
- Most tools search parent folders until they find the file
- `root = true` in `.editorconfig` stops that search
- The Git set covers `.gitignore` and `.gitattributes` in depth

FIVE DOTFILES

```
# .editorconfig
root = true
[*]
indent_style = space
indent_size = 2
end_of_line = lf
insert_final_newline = true
[*.py]
indent_size = 4

# .gitattributes
* text=auto eol=lf
*.png binary
*.sh text eol=lf

# .gitignore
node_modules/
dist/
.env
!.env.example

# .npmrc
save-exact=true
engine-strict=true

# .nvmrc           → 22
# .python-version → 3.13
```

CODE EXPLAINED

→ `[*]` applies to every file; later sections like `[*.py]` override it

→ `* text=auto eol=lf` normalizes line endings in the repo, whatever the OS

→ `!.env.example` re-includes a file a previous pattern ignored

→ `engine-strict` makes npm refuse to install on the wrong Node version

SAME DATA IN…

⇄ **VS Code settings** `"editor.tabSize": 2` · `"files.eol": "\n"` (editor-only, unlike `.editorconfig`)

⇄ **package.json** `"engines": { "node": ">=22" }` (pairs with `.nvmrc`)

GOTCHA

! `.gitignore` does nothing for files already tracked; `git rm --cached` them first

! VS Code needs the EditorConfig extension; JetBrains IDEs read `.editorconfig` out of the box

DOTFILE REFERENCE

FILE	READ BY	FORMAT
`.editorconfig`	Editors / IDEs	INI-like
`.gitignore`	Git	Glob patterns
`.gitattributes`	Git	pattern attr=value
`.npmrc`	npm / pnpm	key=value
`.nvmrc`	nvm, fnm, Volta	Version string
`.python-version`	pyenv, uv	Version string
`.prettierrc`	Prettier	JSON / YAML
`.dockerignore`	Docker build	Glob patterns

.GITIGNORE PATTERNS

PATTERN	MATCHES
`*.log`	Any .log file, any folder
`/build`	build at the repo root only
`dist/`	Folders named dist
`**/tmp`	tmp at any depth
`!keep.log`	Re-include
`# comment`	Comment line

⚠ COMMON MISTAKES

- Committing `node_modules` or build output because `.gitignore` was added too late
- Mixed CRLF / LF line endings because there's no `.gitattributes`
- Different Node versions per developer with no `.nvmrc`

✓ TIPS & BEST PRACTICES

- Start new repos with GitHub's `.gitignore` templates for your stack
- Add `.editorconfig` even if you also use Prettier; it covers files Prettier skips
- Pin runtime versions in dotfiles and in CI from the same value

QUICK SUMMARY

- `.editorconfig` — indentation and line endings
- `.gitignore` / `.gitattributes` — what Git skips and normalizes
- `.npmrc` — npm settings
- `.nvmrc` / `.python-version` — pin the runtime

Validating Config

CATCH BAD CONFIG BEFORE IT SHIPS

A **JSON Schema** describes which keys a config file may have and what type each value must be. Despite the name, editors and tools apply it to **YAML and TOML too**. **SchemaStore.org** hosts schemas for hundreds of common files (workflows, compose, tsconfig, pyproject), and VS Code, JetBrains IDEs and Taplo use it automatically. In CI, `check-jsonschema`, `yamllint`, `taplo check` and `actionlint` fail the build on invalid config.

NOTES

- Many files get a schema from SchemaStore by file name with no setup
- Point a file at a schema with `$schema` (JSON), a modeline comment (YAML) or `#:schema` (Taplo)
- Validate your **own** app config with a schema or a typed loader (Pydantic, Zod)
- The JSON set covers writing schemas

LINKING FILES TO SCHEMAS

```
# JSON: a "$schema" key
{ "$schema": "https://json.schemastore.org/package.json" }

# YAML: modeline comment (Red Hat YAML extension)
# yaml-language-server: $schema=https://json.schemastore.org/github-workflow.json
name: CI

# TOML: Taplo directive at the top of the file
#:schema ./schemas/app-config.schema.json
[server]
port = 8080

# CI checks
check-jsonschema --builtin-schema vendor.github-workflows .github/workflows/*.yml
check-jsonschema --schemafile schemas/app.json config/*.yaml
yamllint -d relaxed .
taplo check pyproject.toml
actionlint
```

CODE EXPLAINED

- → The modeline comment works in any editor using the YAML language server
- → `check-jsonschema` validates JSON, YAML and TOML files against a schema
- → `--builtin-schema` ships common SchemaStore schemas offline
- → `yamllint` checks YAML style and traps; schemas check structure

SAME DATA IN…

- ⇄ **JSON Schema in YAML** `type: object` / `required: [port]` / `properties: {port: {type: integer}}`
- ⇄ **Pydantic** `class Settings(BaseSettings): port: int = 8080`

GOTCHA

- ! A schema only catches what it describes; unknown keys pass unless the schema sets `additionalProperties: false`
- ! Editors may cache SchemaStore schemas; a brand-new key can show as an error until they update

VALIDATORS

TOOL	CHECKS
`check-jsonschema`	JSON / YAML / TOML against a schema
`yamllint`	YAML syntax, style, truthy / octal traps
`taplo check`	TOML syntax + schema
`actionlint`	GitHub workflows, incl. expressions
`hadolint`	Dockerfiles
`docker compose config -q`	Compose file validity

EDITOR SUPPORT

EDITOR	HOW
`VS Code`	JSON built in; Red Hat YAML; Even Better TOML (Taplo)
`JetBrains IDEs`	JSON / YAML / TOML schemas built in
`Neovim`	jsonls / yamlls / taplo + SchemaStore.nvim
`Any (CLI)`	check-jsonschema in pre-commit

⚠ COMMON MISTAKES

- Relying on CI failures to find a typo a schema would have shown in the editor
- Writing a schema without `additionalProperties: false`, so misspelled keys pass
- Validating config only in CI, not when the app starts

✓ TIPS & BEST PRACTICES

- Add the YAML modeline to files SchemaStore can't match by name
- Run validators in pre-commit so mistakes never reach a branch
- Generate the schema from your typed config class (Pydantic `model_json_schema()`)

QUICK SUMMARY

- JSON Schema — works for JSON, YAML and TOML
- SchemaStore — schemas for common files
- `$schema` / modeline / `#:schema` — link a file
- `check-jsonschema`, `yamllint`, `taplo`, `actionlint` — CI checks

Sheet 20

Converting & Querying Config

jq · yq · dasel · taplo · format conversion · reading a key in scripts · round-tripping and comment loss

READ AND CHANGE CONFIG FROM THE COMMAND LINE

Shell scripts and CI often need one value from a config file or a quick format change. **jq** queries JSON; **yq** (mikefarah) uses the same style of expressions on YAML, JSON, TOML, XML and `.properties`, and converts between them; **dasel** and **taplo** do similar jobs. Converting is easy, but most tools **drop comments** and reformat, so convert copies, not the files people edit.

NOTES

- `yq` in this sheet is mikefarah/yq v4 (Go); a different Python `yq` wraps jq
- `-r` (jq) / plain output (yq) prints strings without quotes
- `-i` edits a file in place (yq)
- ruamel.yaml and tomlkit edit files and keep comments, from code

ONE-LINERS

```
jq -r '.version' package.json                  # 1.4.0
jq '.scripts | keys' package.json              # ["build","dev","lint","test"]
yq '.services.db.image' compose.yaml           # postgres:17
yq -p toml '.project.version' pyproject.toml   # 1.4.0
yq -p toml -o json '.' pyproject.toml          # TOML → JSON
yq -o yaml '.' tsconfig.json                   # JSON → YAML
yq -p props -o yaml '.' app.properties         # .properties → YAML
yq -i '.services.api.ports[0] = "9090:3000"' compose.yaml
taplo get -f pyproject.toml project.version    # 1.4.0
VERSION=$(jq -r .version package.json)         # use in a script
git diff --exit-code package.json              # fail CI if a step changed it
```

CODE EXPLAINED

- → `-p` sets the input format and `-o` the output format for yq
- → `yq -i` keeps most comments in YAML, but conversions between formats lose them
- → `jq -r` matters in scripts; without it you get `"1.4.0"` with quotes
- → Both tools return non-zero on parse errors, so they double as quick validators

SAME DATA IN…

- ⇄ **JSON query** `jq '.compilerOptions.strict' tsconfig.json` (fails on comments; strip them or use yq)
- ⇄ **Python** `python -c "import tomllib;print(tomllib.load(open('pyproject.toml','rb'))['project']['version'])"`

GOTCHA

- ! `jq` fails on JSONC files like `tsconfig.json`; `yq -p json` is more forgiving
- ! Round-tripping TOML through JSON loses date types and inline-table style

TOOLS

TOOL	READS	INSTALL
`jq`	JSON	brew / apt / winget install jq
`yq (mikefarah)`	YAML, JSON, TOML, XML, props, CSV	brew install yq
`dasel`	JSON, YAML, TOML, XML, CSV	brew install dasel
`taplo`	TOML	cargo / npm / brew
`xq / xmllint`	XML	with yq / libxml2

EXPRESSION BASICS (JQ / YQ)

EXPRESSION	MEANS
`.a.b`	Nested key
`.items[0]`	First element
`.items[]`	Each element
`.items | length`	Count
`select(.enabled)`	Filter
`keys`	Object keys

⚠ COMMON MISTAKES

- Converting a hand-edited YAML file to JSON and back, losing every comment
- Installing the Python `yq` and getting different flags from the examples
- Parsing config with `grep` / `sed` and breaking on the first reformat

✓ TIPS & BEST PRACTICES

- Read values with `jq` / `yq` in CI instead of hard-coding versions twice
- Use `yq -i` for small automated edits to YAML; review the diff
- Pipe through `yq -P` to pretty-print any supported format

QUICK SUMMARY

- `jq` — query JSON
- `yq -p X -o Y` — convert formats
- `yq -i` — edit in place
- conversions drop comments — edit originals by hand

Sheet 21

Quick Reference — Syntax Side by Side

scalars · null · lists · maps · comments · multi-line text · JSON / YAML / TOML side by side · INI / `.properties` / `.env`

SCALARS (01, 03, 12)

VALUE	JSON	YAML	TOML
String	"text"	text / "text"	"text" / 'text'
Integer	42	42	42, 1_000
Float	3.14	3.14	3.14, inf, nan
Boolean	true	true (1.1: yes, on…)	true
Null	null	null / ~ / (empty)	(omit the key)
Date	"2026-11-03" (string)	2026-11-03	2026-11-03 (native)

LISTS & MAPS (03–05)

SHAPE	JSON	YAML	TOML
List	[1, 2]	- 1 / - 2 or [1, 2]	[1, 2]
Map	{"a": 1}	a: 1	a = 1 / [table]
Nested map	{"a": {"b": 1}}	a: {b: 1}	[a] b = 1 / a.b = 1
List of maps	[{"n": 1}]	- n: 1	[[items]] n = 1
Inline map	{"x": 1}	{x: 1}	{ x = 1 }

COMMENTS & TEXT (06, 11)

FEATURE	JSON	YAML	TOML
Comment	✖ (JSONC: //)	# text	# text
Multi-line	"a\nb"	| keeps lines · > folds	"""…"""
Raw string	✖	'single' (no escapes)	'literal'
Escapes	\n \" \uXXXX	in "double" only	in "basic" only

FLAT FORMATS (07–09)

FEATURE	INI	.PROPERTIES	.ENV
Pair	key = value	key=value	KEY=value
Group	[section]	dotted.keys	PREFIX_ names
Comment	; #	# !	#
Types	Strings	Strings	Strings
Continuation	Indent (configparser)	\ at end	Quoted newline

TOP TRAPS (04, 08, 12)

FORMAT	TRAP
YAML	NO → false, 1.10 → 1.1
TOML	Table defined twice
JSON	Trailing comma
.properties	No inline comments
.env	Real env var overrides it
INI	% needs %% (configparser)

YAML STRINGS (12)

STYLE	EXAMPLE	RESULT
Plain	name: Bob	Bob
Double	"a\tb"	escapes work
Single	'it''s'	it's, no escapes
Literal |	run: |	keeps newlines
Folded >	desc: >	joins lines
Strip |-	key: |-	no final newline

TYPE CHANGES WHEN CONVERTING (01, 20)

FROM → TO	WATCH FOR
TOML → JSON	Dates become strings
YAML 1.1 → JSON	yes / no became booleans
JSON → TOML	null has no TOML form
Any → .env / .properties	Everything becomes a string
JSONC → JSON	Comments dropped
YAML → anything	Anchors expanded, comments lost

TOML NUMBERS & DATES (03, 06)

WRITE	TYPE
1_000	Integer 1000
0xFF / 0o755 / 0b1010	Integer (hex / octal / binary)
6.02e23	Float
inf / nan	Float
2026-11-03T08:00:00Z	Offset date-time
2026-11-03	Local date
07:30 (1.1)	Local time

Sheet 22

Quick Reference — Common Config Files

file → format → owning tool → key settings · where each file lives · which set covers it

JAVASCRIPT & WEB (13, 14, 18)

FILE	FORMAT	KEY SETTINGS	SET
package.json	JSON	scripts, dependencies, type	Node
tsconfig.json	JSONC	strict, module, paths	JS / TS
vite.config.ts	TS	plugins, resolve.alias	Vite
.npmrc	key=value	save-exact, registry	Node
.prettierrc	JSON / YAML	semi, singleQuote	—
eslint.config.js	JS	rules, ignores	—
.nvmrc	version	22	Node

PYTHON & JAVA (08, 15)

FILE	FORMAT	KEY SETTINGS	SET
pyproject.toml	TOML	[project], [tool.*]	Python
.python-version	version	3.13	Python
setup.cfg / tox.ini	INI	legacy tool config	Python
application.properties / .yml	props / YAML	server.port, spring.*	Spring
gradle.properties	props	org.gradle.jvmargs	Gradle
pom.xml	XML	dependencies, plugins	Maven

DEVOPS & CI (16, 17)

FILE	FORMAT	KEY SETTINGS	SET
compose.yaml	YAML	services, ports, volumes	Docker
Dockerfile	Dockerfile	FROM, RUN, COPY	Docker
.github/workflows/*.yml	YAML	on, jobs, steps	GitHub
k8s manifests	YAML	apiVersion, kind, spec	Kubernetes
.env	KEY=value	secrets, URLs	this set
.dockerignore	globs	node_modules, .git	Docker

REPO & EDITOR (18, 19)

FILE	FORMAT	KEY SETTINGS	SET
.gitignore	globs	build output, .env	Git
.gitattributes	pattern attr	text=auto eol=lf	Git
.editorconfig	INI-like	indent_style, end_of_line	this set
.vscode/settings.json	JSONC	formatOnSave	VS Code
.vscode/mcp.json	JSON	servers	VS Code
.github/copilot-instructions.md	Markdown	repo rules for AI	VS Code

LOADING CONFIG IN CODE (03, 07–12)

FORMAT	PYTHON	JAVASCRIPT	JAVA
JSON	json	JSON.parse	Jackson
TOML	tomllib	smol-toml	Jackson TOML / tomlj
YAML	yaml.safe_load	yaml	SnakeYAML
INI	configparser	ini	Commons Configuration
.properties	jproperties	properties-reader	java.util.Properties
.env	python-dotenv	--env-file / dotenv	dotenv-java
JSONC / JSON5	json5	jsonc-parser / json5	Jackson features

VALIDATE & QUERY (19, 20)

TASK	COMMAND
Read a JSON key	jq -r '.version' package.json
Read a TOML key	yq -p toml '.project.version' pyproject.toml
Edit YAML in place	yq -i '.a.b = 1' file.yaml
Schema check	check-jsonschema --schemafile s.json f.yaml
Lint YAML	yamllint .
Lint workflows	actionlint
Compose merged	docker compose config